

Loan Default and Loss Prediction

A Two-Stage Machine Learning Framework

Group 4: Kyle Bartlett, Shahbaz Shaikh, Bhavya Jeevani Thandu

April 29, 2026

Contents

1	Group Members and Contributions	3
2	Executive Summary	4
3	Project Goal and Business Context	4
4	Data Overview	4
5	Data Preprocessing	5
6	Exploratory Dimensionality Reduction	6
7	Modeling Strategy	7
8	Stage 1: Default Classification Results	7
9	Stage 2: Loss Severity Regression Results	8
10	Final Prediction Framework	10
11	Final Test Prediction Summary	10
12	Submission File Output	11
13	Discussion	12
14	Limitations and Future Work	12
15	Conclusion	14

16 Appendix: Selected R Code Chunks	15
16.1 Target Creation	15
16.2 Train-Validation Split	15
16.3 XGBoost Classifier	15
16.4 XGBoost Regressor	16
16.5 Final Expected-Loss Prediction	16

1 Group Members and Contributions

Group Member	Contribution
Shahbaz Shaikh	Led the R Markdown implementation, including data preprocessing, model construction, model comparison, result interpretation, and final report preparation. Developed the two-stage machine learning framework using classification and regression models, generated the final <code>submission.csv</code> file, and prepared the analytical interpretation of results.
Kyle Bartlett	Supported exploratory data analysis and reviewed the financial/business context of the loan default problem. Assisted with understanding the structure of the training and test datasets, reviewing the loss distribution, and contributing to the explanation of why loan loss prediction is relevant for credit risk and underwriting decisions.
Bhavya Thandu	Assisted with model evaluation, results validation, and presentation of findings. Contributed to comparing classification and regression model outputs, reviewing performance metrics such as AUC and MAE, and helping organize the final insights and conclusions for the project report.

2 Executive Summary

This project builds a machine learning framework for predicting expected loan loss. The target variable, `loss`, is highly zero-inflated: most loans have zero loss, while only a smaller share of loans have positive loss. Because of this structure, the project uses a two-stage framework instead of a single direct regression model.

The final prediction is calculated as:

$$\text{Expected Loss} = P(\text{Default}) \times \max(\text{Predicted Loss Severity}, 0)$$

Stage 1 predicts whether a loan is likely to default, where default is defined as `loss > 0`. Stage 2 predicts loss severity only among positive-loss loans. The final output is saved in the required `id`, `loss` submission format.

The main modeling result is that XGBoost performs best in both operational stages of the framework. In Stage 1, XGBoost achieves the strongest AUC, meaning it ranks risky/defaulting loans better than the linear alternatives. In Stage 2, XGBoost Regressor achieves the lowest MAE, meaning it predicts positive loss severity with the smallest average absolute error among the severity models tested.

3 Project Goal and Business Context

The goal is to predict the `loss` value for each row in the test dataset. The test set does not include the true `loss` column, so the trained model must generate one expected-loss prediction for every test record.

This problem is important for credit risk because lenders need more than a simple good/bad borrower label. A loan can have a low default probability but high possible severity, or a high default probability with moderate severity. The two-stage framework supports this business interpretation by separating probability of default from conditional loss severity.

From a business analytics perspective, the output is useful because it produces a continuous expected-loss estimate rather than only a binary default prediction. This means the model can support ranking, monitoring, and portfolio-level risk summaries. A decision-maker can identify not only which loans are likely to default, but also which loans may contribute the highest expected financial loss.

4 Data Overview

The raw files show:

Dataset	Rows	Raw Columns	Explanation
Training file	80,000	763	Includes <code>Unnamed: 0</code> , <code>id</code> , 760 predictors, and target <code>loss</code> .
Test file	25,471	762	Includes <code>Unnamed: 0</code> , <code>id</code> , and the same 760 predictors. It does not include <code>loss</code> .

After removing the extra index-style column, the identifier column, and the target where appropriate, the final modeling feature set contains 760 usable predictor features.

The difference between raw column counts and modeling feature counts is important. The raw training file contains 763 total columns because it includes the index-like column `Unnamed: 0`, the identifier `id`, the 760 predictor columns, and the target column `loss`. The raw test file contains 762 total columns because it has the same index-like column

and identifier, but it does not contain `loss`. After cleanup, both train and test have the same 760 predictor columns available for modeling.

Metric	Value
Training Rows	80,000
Test Rows	25,471
Predictor Features Used for Modeling	760
Default Rate (<code>loss > 0</code>)	9.22%
Zero-Loss Rate	90.78%

The feature labels go up to `f777`, but the dataset does not contain every label from `f1` to `f777`. There are 17 skipped feature labels: `f2`, `f10`, `f11`, `f12`, `f230`, `f271`, `f274`, `f462`, `f463`, `f473`, `f474`, `f528`, `f597`, `f602`, `f603`, `f605`, and `f670`. This is why the highest label is `f777`, but only 760 actual predictor columns are present.



Figure 1: Training Loss Distribution

The loss distribution confirms that the target is highly zero-inflated. This motivates treating the project as both a classification problem and a regression problem.

The zero-inflation is the central modeling challenge. If the full loss target were modeled directly with a single regression model, the model would see a large majority of zero values and could become biased toward predicting very small losses. That would look reasonable for the majority class but would weaken performance on the positive-loss cases that matter most for credit-risk decision-making.

5 Data Preprocessing

The preprocessing pipeline was designed to prepare the data for all model families while preventing data leakage.

The major preprocessing steps were:

- Removed the extra CSV index column `Unnamed: 0`.
- Stored `id` separately for final submission.
- Removed `loss` from the feature matrix because it is the target variable.

- Created the binary default target where `default = 1` if `loss > 0`, otherwise 0.
- Aligned test features with training features.
- Encoded non-numeric columns using mappings learned from the training split.
- Imputed missing values using median imputation.
- Standardized features using centering and scaling.
- Cleaned non-finite values before XGBoost modeling.

Median imputation was used because financial variables can be skewed and affected by outliers. Standardization was used because regularized linear models, Ridge, Elastic Net, and PLS are sensitive to feature scale.

The preprocessing design also keeps the validation and test sets honest. Encoding mappings, imputation values, and scaling parameters are learned from the training data and then applied to validation/test records. This avoids using information from the validation or test data while fitting preprocessing steps, which would create data leakage and overstate model performance.

The `id` column is retained only for final submission. It is removed from the model inputs because it is a record identifier, not a behavioral or financial predictor. The `loss` column is removed from the feature matrix because it is the answer being predicted. Keeping `loss` as an input would make the model artificially strong and invalid.

6 Exploratory Dimensionality Reduction

The R Markdown file includes an exploratory comparison of PCA and PLS. This section is not the final model selection step; it checks whether reduced feature representations can support loss prediction.

Method	MAE
PLS	1.410756
PCA	1.411261

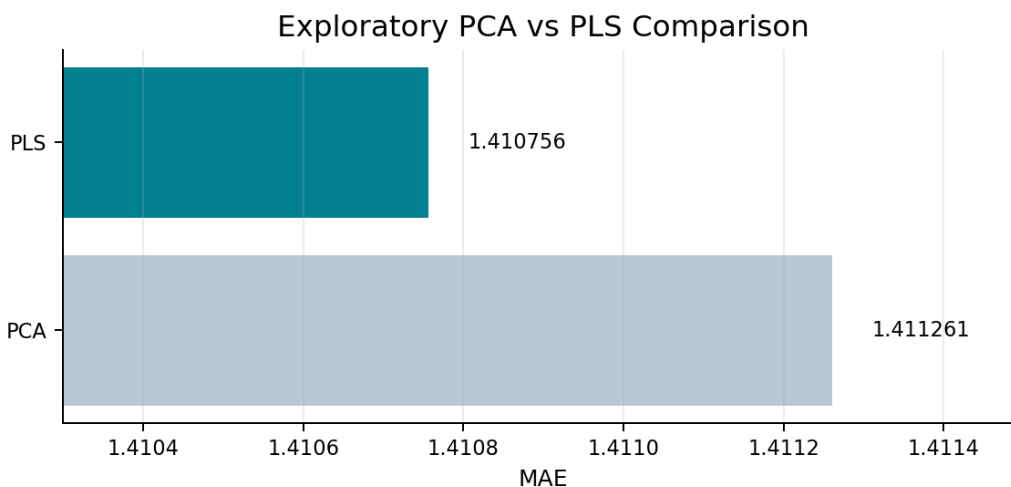


Figure 2: Exploratory PCA vs PLS Comparison

PLS slightly outperformed PCA in this exploratory comparison. The difference is very small, so this section is treated as supporting analysis rather than the final modeling choice.

PCA and PLS answer different questions. PCA is unsupervised: it creates components that explain variance in the feature matrix without looking at the target. PLS is supervised: it creates components that are more directly related to the response variable. Because this project has many anonymized and potentially correlated predictors, it is useful to check whether lower-dimensional representations can preserve predictive signal. However, the final two-stage model is still selected using the dedicated Stage 1 classification and Stage 2 severity-model comparisons.

7 Modeling Strategy

The final modeling strategy uses two stages:

Stage	Task	Target	Metric
Stage 1	Classification	<code>default = 1 if loss > 0</code>	AUC
Stage 2	Regression	Positive <code>loss</code> amount among defaulted loans	MAE

AUC is appropriate for Stage 1 because the default class is imbalanced and we care about ranking risky loans above safer loans. MAE is appropriate for Stage 2 because it measures the average absolute prediction error in the original loss scale.

This framework mirrors the expected-loss logic used in credit risk. The first model estimates the likelihood of a loss event. The second model estimates how severe that event may be. Multiplying the two gives a final expected-loss value that can be compared across loans. This is more useful than a binary label because it supports ranking loans by expected financial impact.

8 Stage 1: Default Classification Results

Stage 1 predicts the probability that a loan will generate any positive loss. The models compared in the R Markdown and HTML output are shown below.

Model	Validation AUC
XGBoost Classifier	0.712861
Elastic Net Logistic	0.705165
Logistic Regression	0.702779
Random Forest Benchmark	0.621714

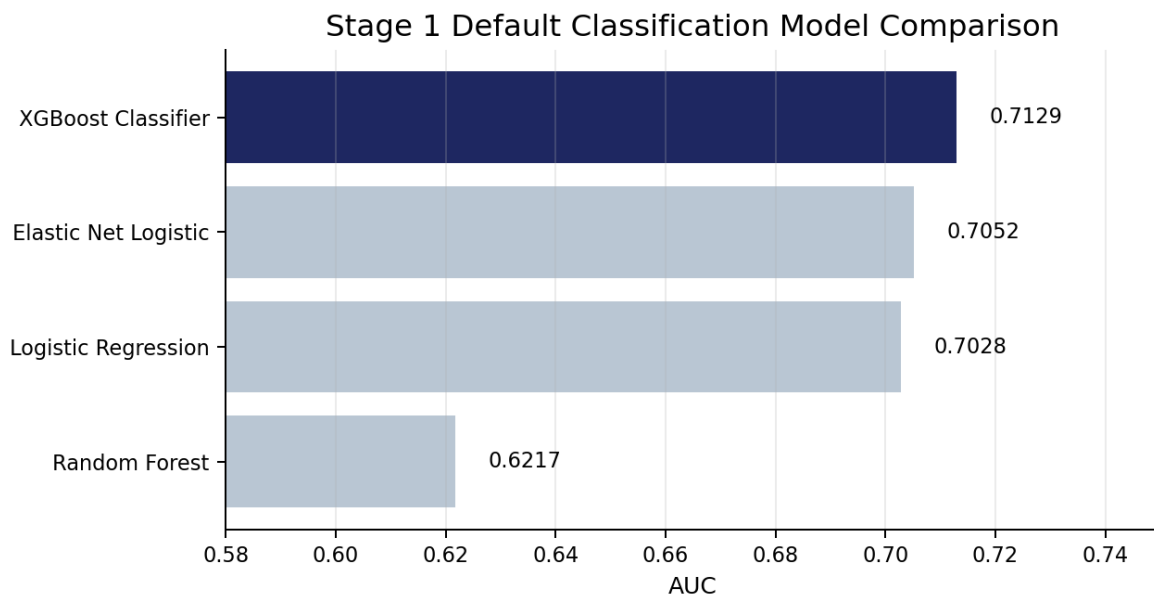


Figure 3: Stage 1 Default Classification Model Comparison

XGBoost achieved the highest AUC, so it was selected as the Stage 1 model for generating default probabilities. Elastic Net and Logistic Regression were close behind, which suggests that the default signal includes meaningful linear structure. Random Forest performed worse in this run, indicating that bagging did not rank default risk as effectively as XGBoost boosting.

Each Stage 1 model served a different purpose:

- Logistic Regression provides a simple and interpretable baseline.
- Elastic Net Logistic Regression adds regularization, which is useful when there are many features and possible correlation among predictors.
- XGBoost Classifier provides a nonlinear boosted-tree model that can capture interactions and nonlinear default patterns.
- Random Forest was included as an additional tree-based benchmark to compare bagging against boosting.

The AUC values show that XGBoost was the strongest classifier, but the closeness between Logistic Regression, Elastic Net, and XGBoost also suggests that some default behavior may be captured by relatively stable linear relationships. XGBoost adds performance by capturing additional nonlinear structure.

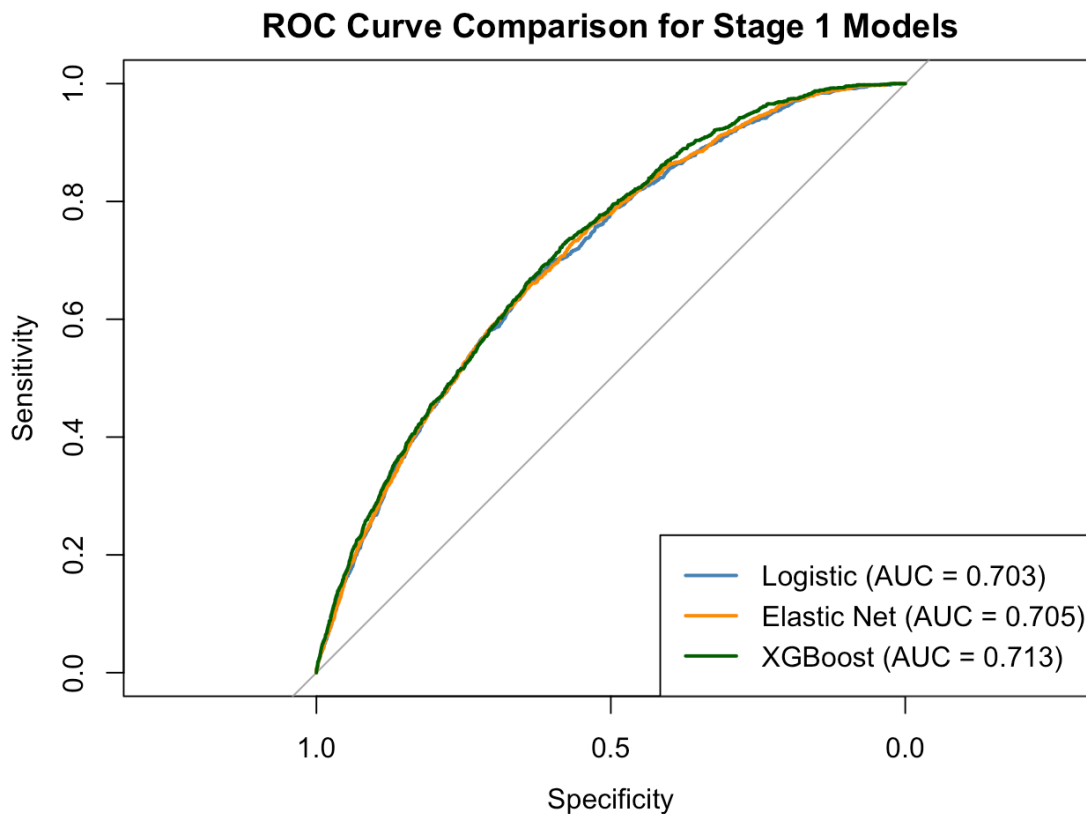


Figure 4: ROC Curve Comparison for Stage 1 Models

The ROC curve shows that the classification models perform better than random ranking. XGBoost has the strongest curve and the highest AUC among the main Stage 1 models.

9 Stage 2: Loss Severity Regression Results

Stage 2 predicts loss severity only for rows where training loss is positive. In the validation split, the Stage 2 data contained:

Stage 2 Split	Rows
Training positive-loss rows	5,926
Validation positive-loss rows	1,453

The Stage 2 model comparison from the rendered R Markdown output is:

Model	Validation MAE
XGBoost Regressor	5.033003
Ridge Regression	5.934699
PLS Regression	6.432512

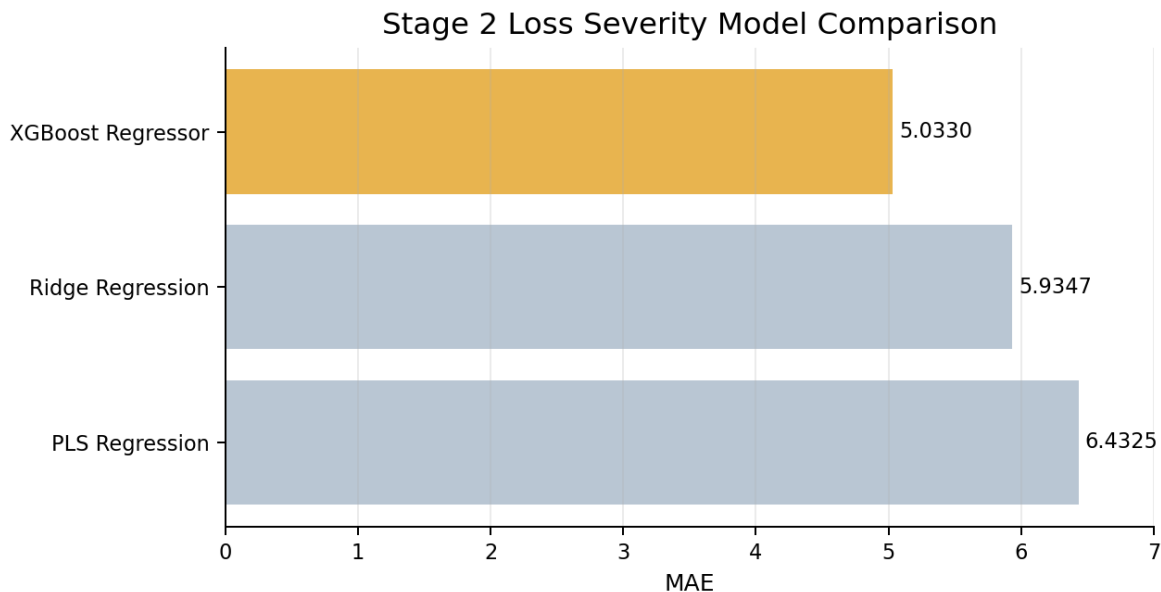


Figure 5: Stage 2 Loss Severity Model Comparison

XGBoost Regressor achieved the lowest MAE and was selected as the Stage 2 model. Ridge Regression provided a regularized linear baseline, while PLS tested a supervised dimensionality-reduction approach. XGBoost performed best because it can capture nonlinear relationships and interactions in the severity data.

This section is separate from Stage 1 because severity modeling is a different statistical problem. Stage 1 asks whether a loss happens. Stage 2 asks how large the loss is, conditional on positive loss. For this reason, the model is trained only on rows where `loss > 0`.

The regression models included:

- Ridge Regression, a regularized linear model that provides a stable baseline for high-dimensional data.
- PLS Regression, a supervised component-based regression method that can help when predictors are numerous and correlated.
- XGBoost Regressor, a nonlinear boosted-tree model designed to learn interactions and more complex severity patterns.

The earlier PCA vs PLS comparison was exploratory and looked at reduced representations for general loss prediction. The final Stage 2 comparison is the operational severity-model comparison because it is trained only on positive-loss observations, which matches the two-stage expected-loss framework.

The actual-versus-predicted plot shows that the model captures the general severity pattern but still has larger errors for high-loss cases. This is expected because extreme financial losses are rare and harder to predict precisely.

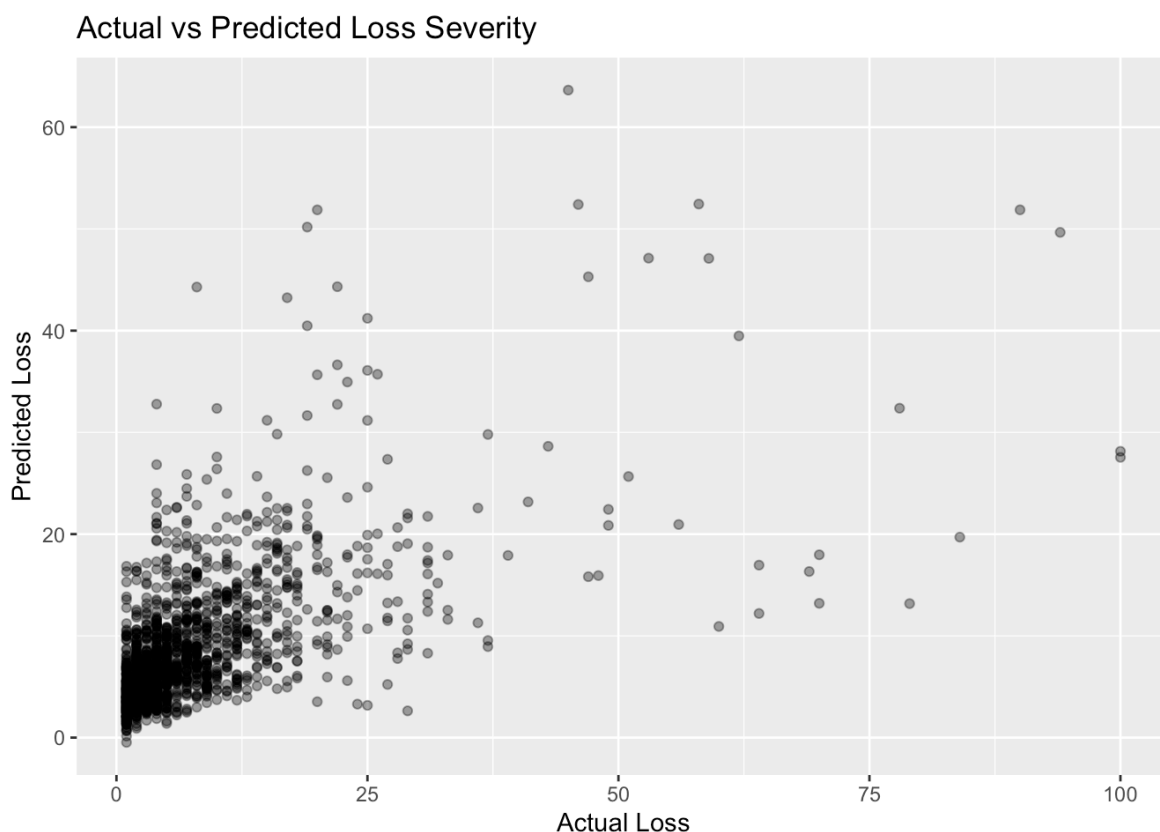


Figure 6: Actual vs Predicted Loss Severity

The scatter plot also shows why MAE is a useful metric here. High-loss observations are uncommon and difficult to estimate exactly, so the model is evaluated by average absolute prediction error rather than exact point matching. The goal is not to perfectly predict every rare extreme value, but to produce a severity estimate that improves expected-loss calculation across the portfolio.

10 Final Prediction Framework

The final model uses XGBoost in both stages:

1. Stage 1 XGBoost Classifier estimates default probability for every test loan.
2. Stage 2 XGBoost Regressor estimates possible loss severity.
3. Negative severity predictions are clipped at zero.
4. The final expected loss is computed as default probability times predicted severity.

The final prediction is produced for every row in the test set. The test set is not assumed to be fully defaulted; its true outcomes are hidden. The model generates an expected-loss estimate for each `id`.

This distinction is important. Stage 2 is trained only on positive-loss rows because that is where severity is observed. But during final scoring, every test observation receives a default probability and a potential severity estimate. The multiplication step adjusts severity by default probability, producing a final expected-loss value for all test rows.

11 Final Test Prediction Summary

The final test-set default summary from the rendered R Markdown output is:

Metric	Value
Total Test Observations	25,471
Predicted Defaults at Threshold 0.5	11,236
Predicted Default Percentage	44.11%
Expected Defaults from Probability Sum	11,359.37
Expected Default Percentage	44.60%

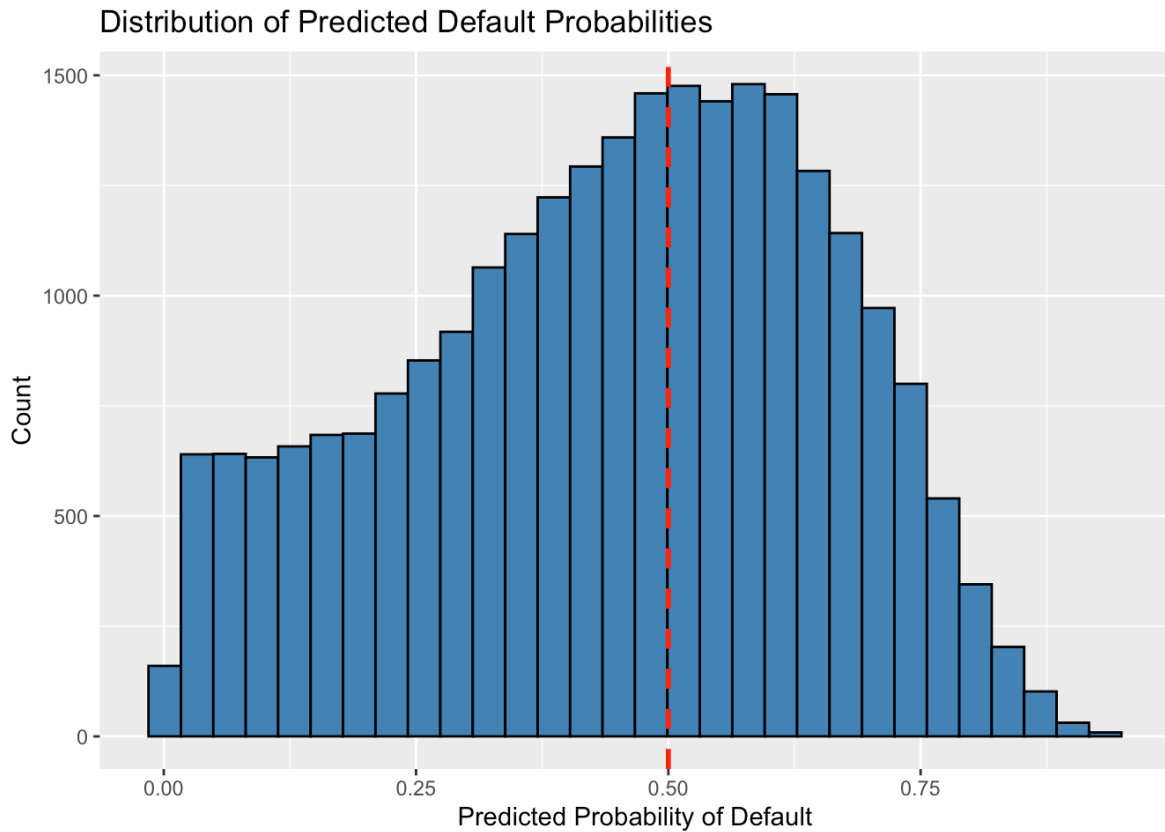


Figure 7: Distribution of Predicted Default Probabilities

The threshold-based count says how many test loans are classified as default if the cutoff is 0.5. The probability-sum estimate is a softer expected count because every loan contributes its probability of default.

The threshold count and probability-sum count answer related but different questions. The threshold count converts each probability into a hard default/non-default decision. The expected-default count sums the probabilities directly, which is often more useful for portfolio-level analysis because it preserves uncertainty.

12 Submission File Output

The prediction file has 25,471 rows and two columns: `id` and `loss`.

id	loss
7933	10.618965
101860	7.551583
62580	1.793808
1760	5.205888

id	loss
48008	5.190197
9308	4.200051
27862	4.801249
87760	5.061059
53334	1.121561
72303	5.529714

Summary statistics for the final prediction file are:

Metric	Value
Total Submission Rows	25,471
Minimum Predicted Loss	0.000000
Average Predicted Loss	4.876936
Median Predicted Loss	4.442246
Maximum Predicted Loss	29.248163
Total Predicted Portfolio Loss	124,220.04

The predicted loss values are mostly concentrated at lower levels, with a smaller number of test loans showing much higher expected loss. This is consistent with the zero-inflated nature of the original training target.

The first rows of the submission file are not actual observed losses; they are model predictions. The file is designed for scoring, so every test-set `id` must appear exactly once with one predicted `loss` value. A predicted loss of zero or near zero means the model estimates very low expected loss for that loan. Higher values indicate loans with higher expected financial impact.

13 Discussion

The two-stage framework is more suitable than a single regression model because the target has two separate behaviors: a large group of loans with zero loss and a smaller group with positive loss. Stage 1 handles default occurrence, while Stage 2 handles loss magnitude among defaults.

XGBoost performed best in both final model stages. It achieved the highest Stage 1 AUC and the lowest Stage 2 MAE. This is reasonable because XGBoost is well suited for high-dimensional tabular data and can learn nonlinear relationships and interactions.

The final expected-loss output has a direct business interpretation. It combines the probability that loss occurs with the estimated size of that loss, creating a loan-level risk score that can support underwriting, monitoring, and portfolio risk analysis.

The model comparisons support the final choice. In Stage 1, XGBoost achieves the highest AUC, meaning it is the strongest model for ranking likely defaults. In Stage 2, XGBoost Regressor achieves the lowest MAE, meaning it is the strongest model for estimating severity among positive-loss loans. The same algorithm performs well in both stages, but the tasks and metrics are different.

The regression section includes both exploratory and final modeling components. PCA and PLS were explored as dimensionality-reduction methods, while Ridge, PLS, and XGBoost were compared in the actual Stage 2 severity modeling setup. This broader comparison gives a clearer view of why the final model uses XGBoost Regressor.

14 Limitations and Future Work

The project has several limitations:

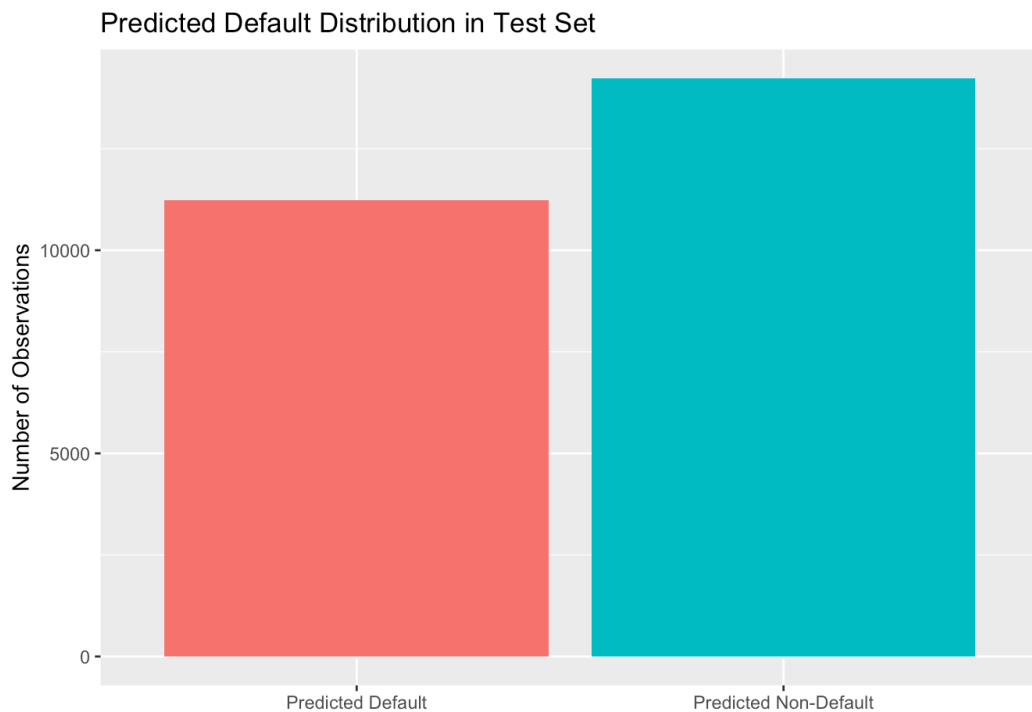


Figure 8: Predicted Default Distribution in Test Set

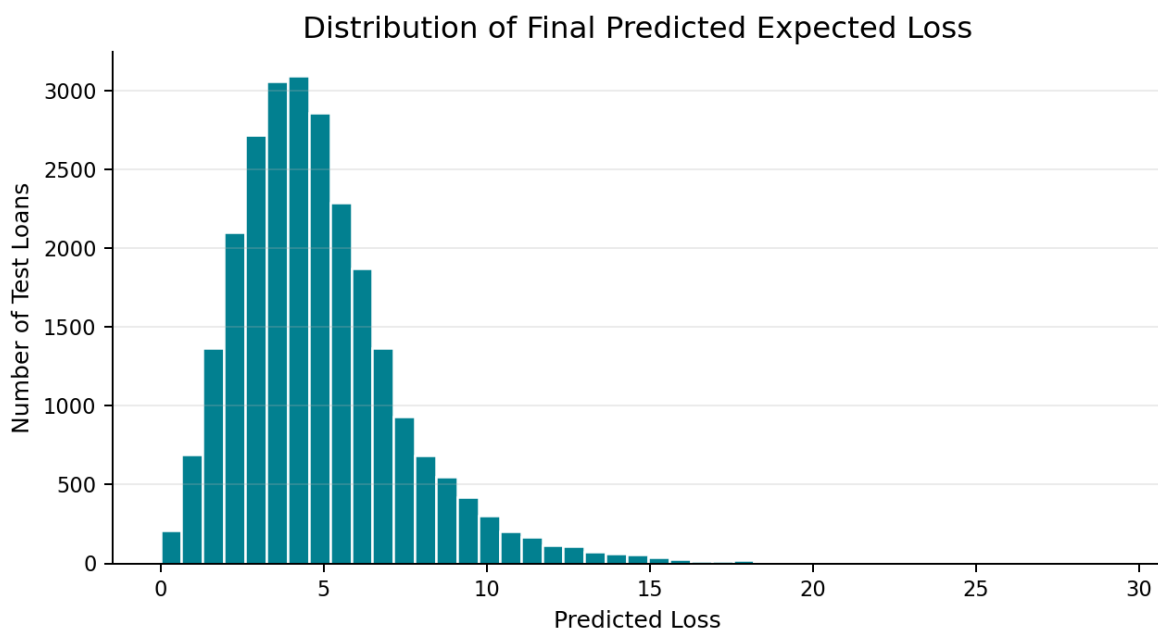


Figure 9: Distribution of Final Predicted Expected Loss

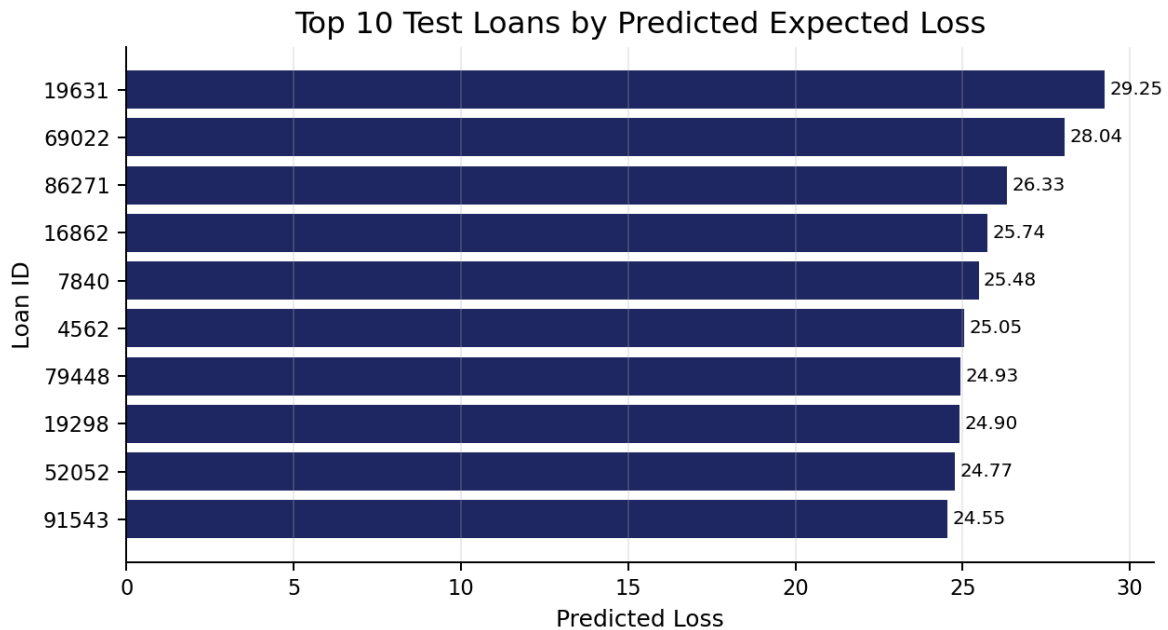


Figure 10: Top 10 Test Loans by Predicted Expected Loss

- The features are anonymized, which prevents domain-specific feature engineering.
- The default class is highly imbalanced.
- Extreme losses are rare, making high-loss severity difficult to predict.
- Further hyperparameter tuning could improve the XGBoost models.
- Model explainability could be expanded using SHAP or LIME.

Future work could compare LightGBM, CatBoost, Tweedie regression, quantile regression, and business-cost based threshold optimization.

Another useful extension would be calibration analysis for the Stage 1 default probabilities. Since the final expected loss depends directly on predicted probabilities, better-calibrated probabilities may improve the business interpretation of portfolio-level expected loss. For Stage 2, future work could also test loss transformations or robust regression methods to better handle rare high-severity observations.

15 Conclusion

This project successfully builds a two-stage expected-loss model for loan prediction. Stage 1 estimates default probability, Stage 2 estimates loss severity, and the final output multiplies both components to generate expected loss for every test observation.

The final model and submission file are consistent with the business objective: estimate expected financial loss at the loan level using historical data and machine learning.

Overall, the project demonstrates the value of framing the problem correctly. Instead of forcing a single model to learn both zero-loss behavior and positive-loss severity, the two-stage approach separates the problem into two interpretable components. This makes the final output easier to explain, easier to evaluate, and more aligned with how credit risk is understood in practice.

16 Appendix: Selected R Code Chunks

The following selected chunks from the RStudio raw file show the core implementation logic.

16.1 Target Creation

```
train_ids <- train_df$id
test_ids  <- test_df$id

y_loss <- train_df$loss
y_default <- ifelse(y_loss > 0, 1, 0)

X_train_raw <- train_df %>% select(-id, -loss) %>% as.data.frame()
X_test_raw  <- test_df %>% select(-id) %>% as.data.frame()
X_test_raw <- X_test_raw[, names(X_train_raw)]
```

16.2 Train-Validation Split

```
split_index <- createDataPartition(y_default, p = 0.8, list = FALSE)

X_train_split_raw <- X_train_raw[split_index, , drop = FALSE]
X_val_split_raw   <- X_train_raw[-split_index, , drop = FALSE]

y_loss_train <- y_loss[split_index]
y_loss_val   <- y_loss[-split_index]

y_default_train <- y_default[split_index]
y_default_val   <- y_default[-split_index]
```

16.3 XGBoost Classifier

```
scale_pos_weight <- sum(y_default_train == 0) / max(sum(y_default_train == 1), 1)

dtrain_cls <- xgb.DMatrix(data = as.matrix(X_train_scaled), label = y_default_train)
dval_cls   <- xgb.DMatrix(data = as.matrix(X_val_scaled), label = y_default_val)

xgb_classifier <- xgb.train(
  params = list(
    objective = "binary:logistic",
    eval_metric = "auc",
    max_depth = 4,
    eta = 0.05,
    subsample = 0.9,
    colsample_bytree = 0.9,
    scale_pos_weight = scale_pos_weight
  ),
  data = dtrain_cls,
  nrounds = 300,
  verbose = 0
)
```

16.4 XGBoost Regressor

```
dtrain_reg <- xgb.DMatrix(data = as.matrix(X_train_loss_only), label = y_train_loss_only)
dval_reg   <- xgb.DMatrix(data = as.matrix(X_val_loss_only), label = y_val_loss_only)

xgb_regressor <- xgb.train(
  params = list(
    objective = "reg:squarederror",
    eval_metric = "mae",
    max_depth = 4,
    eta = 0.05,
    subsample = 0.9,
    colsample_bytree = 0.9
  ),
  data = dtrain_reg,
  nrounds = 400,
  verbose = 0
)
```

16.5 Final Expected-Loss Prediction

```
test_default_probability <- predict(final_xgb_classifier, dtest_full)
test_loss_prediction <- predict(final_xgb_regressor, dtest_full)
test_loss_prediction <- pmax(test_loss_prediction, 0)

final_test_prediction <- test_default_probability * test_loss_prediction

submission_df <- data.frame(
  id = test_ids,
  loss = final_test_prediction
)

write.csv(submission_df, "submission.csv", row.names = FALSE)
```